



Trust but verify. Or don't trust at all.

In **entropic security**¹, the cryptosystem needs to guarantee that the entropy of the message space is **high from the point of view of the adversary**. This paper uses Shannon entropy as the basis for **verifiable unconditional information security which remains secure** against adversaries with unlimited computational power, resources, memory space, and time.

*Entropic security defines and codifies via test case Shannons original Information Theoretic Security definition of a **random key**, to ensure the **secure key** remains **unpredictable** and possesses **high min-entropy from the adversary's perspective**, even after observing the ciphertext, and when the plaintext message **may have zero entropy**. Entropic security does not remove or redefine any of Shannons original requirements within this **Security-Proof**.*

In cryptography and information theory, XORing an **entropic key** (a key with high uncertainty or randomness, contains zero collisions and passes a NetBitTest) with a **zero-entropy value** (a completely predictable constant, like all zeros) results in an **output that retains the exact same entropy as the original key**.

Entropic Security:

- **The Math:** Let K be your entropic key and Z be a string of all zeros (000...0).
- By definition of the XOR ($\oplus$) operation: $K \oplus Z = K$.
- Because the output is identical to K , no information is gained or lost, meaning the **Shannon entropy** $H(K \oplus Z) = H(K)$.

If the entropy drops or changes during the above specific operation, it signals a flaw in the implementation, memory management, or secure key generation.

¹ Nobody knows what *entropic security* really is, so in any cryptographic based debate we will always have the advantage."

The Information Theoretic Security-Proof, for the Medley-One Cipher is based upon information theory as defined via the following six principles:

1. Theoretical Proof...

From Shannons 1949 paper, Communication Theory of Secrecy Systems, that an encryption system has perfect secrecy:

An encryption system has **perfect secrecy** if the conditional probability of any plaintext message given a ciphertext is equal to the unconditional probability of that message ($P(M|C) = P(M)$), meaning the ciphertext yields no additional information about the underlying plaintext

Definition of Perfect Secrecy

- **Independent Distributions:** In Shannon's formulation from "*Communication Theory of Secrecy Systems*" (1949), a cryptosystem achieves perfect secrecy when the a posteriori probability of the message M given the ciphertext C is identical to the a priori probability of M
- **Mathematical Condition:** $P(M = m | C = c) = P(M = m)$ for all messages m and ciphertexts c with non-zero probability. Equivalently, the mutual information between the plaintext and ciphertext is zero: $I(M; C) = 0$

Core Proof Insights & Key Theorem

1. **Equal Likely Plaintexts/Ciphertexts:** Shannon demonstrated that for a system to achieve perfect secrecy, observing the ciphertext $E(K, M) = C$ must leave all possible plaintexts equally likely from the attacker's perspective, regardless of the attacker's computational power.
2. **The Size Bound (Entropy Condition):** Shannon proved that perfect secrecy requires the key space size ($H(K)$, the entropy or uncertainty of the key) to be **at least as large** as the message space size ($H(M)$, the entropy of the message): $H(K) \geq H(M)$.
3. **Implication for Key Length:** If messages are measured in bits and chosen from a space of n bits, the key must contain at least n bits of truly random information ($|K| \geq |M|$). This mathematically validates that the One Cipher is a theoretically unbreakable cipher as it satisfies the core properties of the proof of security namely a variable uniform random key, used only once, and key length $\geq$ information object).

The One Cipher makes the following Proof of Security claims:

- a) $P(M = m | C = c) = P(M = m)$ i.e. seeing a ciphertext doesn't give you any extra information about the plaintext. The probability of seeing a message m after

the ciphertext has been observed is the same as the probability of the message without the ciphertext.

b) $P(C = c \mid M = m_0) = P(C = c \mid M = m_1)$ i.e. the probability of ciphertext c is equally likely for 2 different messages.

c) The Medley KG will always generate a keys:

Key Length: The key must be at least as long as the message ($|K| \geq |M|$).

Uniform Distribution: Each key must be chosen uniformly at random from the key space ($P(K=k) = 1/|K|$).

Single Use: The key must be completely unique and used only once.

$$C = M \oplus K \quad \leftarrow \text{The ciphertext (C) is constructed by XOR-ing the message (M) with a random key (K)}$$

$$p(M = m \mid C = c) = p(M = m) \quad \leftarrow \text{The ciphertext provides no information about the message.}$$

$$I(M; C) = H(M) - H(M \mid C) = 0 \quad \leftarrow \text{Equivalent: zero mutual information between the message and ciphertext}$$

Figure 1 Summary of Claims

In Conclusion

The only mathematically unbreakable encryption scheme requires a key that is the exact same length as the message, used only once, and sampled from a perfectly uniform distribution. Such as the Medley One Cipher.

When Shannon entropy = Shannon min entropy, an attacker cannot predict the next outcome with any higher accuracy than a random guess.

Kerckhoffs's principle states that a cryptographic system must remain secure even if everything about it is public knowledge, except for the secret key. This means that the security of any information system is dependent on the security of the secret key.

No known function $f(x)$ can increase the security or min-entropy or uniform distribution of the secret key. Claude Shannon mathematically reinforced this principle and introduced the concept of perfect secrecy using the XOR One-Time Pad (OTP).

According to Kerckhoffs's principle:

- An attacker knows the One Cipher (XOR algorithm) is being used.
- An attacker intercepts the Ciphertext (C).
- . Despite knowing the exact $f(x)$ as the One Cipher (Vernan cipher), the attacker cannot learn anything about the Plaintext (P) without the Key (K).

Knowing the One Cipher algorithm (Kerckhoffs's Principle) gives the attacker no advantage.

A bit stream with maximum Shannon entropy (ergo maximum information content) is nature aggregated random noise. It contains the maximum possible “information content” in terms of bits needed to represent it, it is computationally incompressible, it also has no meaningful or useful information in the conventional sense.

Claude Shannon proved that achieving information-theoretic perfect secrecy demands that every key bit of a secure bit stream is chosen uniformly at random, meaning the min-entropy per bit is maximal.

The Medley Key Generator must be **self or independently verified** (using the supplied source code or AI prompts) to produce Shannon min-entropy output with zero collisions and Next Bit Test pass.

Proving:

- The knowledge of the Cyphertext is not helpful at all!
- The result is independent of the capability of any attacker or computational resources.
- The Medley One Cipher, based upon XOR as implemented, and has an output with proven and verifiable Shannon min-entropy = entropy, in accordance with above is provably secure against all attacks (known and unknown).

The above proof requires a "cryptographically secure" and verified Shannon min-entropy, bit stream (CSRBG) as the proof, in order to implement a truly secure one-time pad.

The Medley Key Generator (KG) is one such example of a Cryptographically Secure Non-Deterministic Random Bit Generators (CSNRBG).

It should be noted that a CSRBG cannot be computed Soley from a TRNG seed, as this would produce a CSPRNG key, which would not meet the requirements of the above proof.

2. Verifiable Proof...

Self-verifying cryptography is the safest mindset when dealing with digital security, never trust 'so called experts.

"Trust but verify. Or don't trust at all..."

In order to achieve Information Theoretic Security, the outputs from each cipher must be validated against the intersection of Shannon min-entropy and Shannon entropy to ensure a perfect uniform distribution.

Only after a Pass result (validation) can the specific instance of the cipher output be considered as Information Theoretic Secure.

No cipher can be generically considered secure unless each and every cipher output passes validation.

The following code is provided to allow a binary self-validation of any claimed proof of security against any cipher output data stream.

```
public static (double ShannonEntropy, double MinEntropy) Calculate(byte[] data)
{
    if (data == null || data.Length == 0)
        return (0.0, 0.0);

    int[] counts = new int[256];
    foreach (byte b in data)
    {
        counts[b]++;
    }

    double shannonEntropy = 0.0;
    int maxCount = 0;
    double len = data.Length;

    foreach (int count in counts)
    {
        if (count > 0)
        {
            if (count > maxCount) maxCount = count;
            double p = count / len;
            // Shannon entropy summation: -p * log2(p)
            shannonEntropy -= p * Math.Log2(p);
        }
    }

    double minEntropy = -Math.Log2((double)maxCount / len);

    return (shannonEntropy, minEntropy);
}
```

// Test vectors, for range limits.

// Entropy = 8

```
byte[] allValues = Enumerable.Range(0, 256).Select(i => (byte)i).ToArray();
```

// Entropy = 0

```
{ "Test Zero Entropy Bytes", new byte[size]},  
{ "Test Maximum Entropy Bytes", allValues},
```

// Birthday attack or collisions:

```
public static int CalcCollisions(byte[] data)  
{  
    // Group by value and filter groups with more than 1 element  
    var collisions = data.GroupBy(x => x)  
                        .Where(g => g.Count() > 1)  
                        .Select(g => g.Key);  
    return collisions.Count();  
}
```

AI Versions

1. Prompt: "calculate Shannon entropy and min entropy in bits per 8 bits for this hex encoded byte array
2C9D8EFC726469F6974DF4EC05C9292360CDE3C66F5510175BF7C13E532481A8"
2. Prompt: "verify maximum number of collisions is zero for this hex encoded byte array
2C9D8EFC726469F6974DF4EC05C9292360CDE3C66F5510175BF7C13E532481A8"
3. To test any TRNG/QRND, classic AES/BATTON or PQC use prompt: calculate Shannon min entropy in bits per 8 bits for this hex encoded byte array <add hex encoded output stream>

Example Results

Range Tests

Test Zero Entropy Bytes: Shannon-entropy 0.0000 min-entropy 0.0000

Test Maximum Entropy Bytes: Shannon-entropy 8.0000 min-entropy 8.0000

Entropy Tests

Medley One Cipher Bytes: Shannon-entropy 5.0000 min-entropy 5.0000

AESGCM Cipher Text Bytes: Shannon-entropy 4.8125 min-entropy 4.0000

Lattice Ring-LWE Cipher Text Bytes: Shannon-entropy 4.9375 min-entropy 4.0000

Collision Tests

AESGCM Collisions 3.0000

Medley One Cipher Collisions 0.0000

Entropic Security Check

In cryptography and information theory, **entropic security** dictates how leakage behaves when high-entropy data undergoes transformations. When you perform a bitwise One Cipher **transformation** between a highly random variable (the entropic key) and a completely deterministic, invariant block (zero entropy, such as a buffer populated entirely with 0x00 or a single constant string), the resulting output's Shannon entropy must exactly match the initial key's entropy. This happens because One Cipher encryption with a static bias shifts the distribution deterministically without changing its underlying probability vectors or structure.

```
byte[] key = RandomNumberGenerator.GetBytes(size);
byte[] zeroEntropyBuffer = EntropyCalculator.ZeroEntropyByteArray(key.Length);
byte[] xorResult = OneCipher.Encrypt(key, zeroEntropyBuffer).Skip(16).Take(key.Length).ToArray(); // remove One Cipher 16 byte authentication tag
bool p = EntropyCalculator.EntropicCheck(key, xorResult);
Debug.WriteLine($"OneCipher->Entropic Security Pass: {(p ? "Yes" : "No")}");
```

Figure 2 One Cipher Entropic Security Test Case

```
// AES
byte[] nonce = new byte[12];
byte[] tag = new byte[16];
byte[] ciphertext = new byte[key.Length];
using (var aesGcm = new AesGcm(key, 16))
{
    aesGcm.Encrypt(nonce, zeroEntropyBuffer, ciphertext, tag);
}
p = EntropyCalculator.EntropicCheck(key, ciphertext);
Debug.WriteLine($"AES->Entropic Security Pass: {(p ? "Yes" : "No")}");
```

Figure 3 AES Entropic Security Test Case

Entropic Security Test Results

One Cipher->Entropic Security Pass: Yes

AES->Entropic Security Pass: No

Security Conjecture

If any cipher claims to be secure, the tested output using the above functions, shall have Shannon min-entropy within 90% of Shannon entropy to be considered uniform and hence secure. Scores which deviate by more than 90% may reveal hidden patterns, biases, or repeating sequences that algorithms or bad actors can exploit. The 10% variation was selected based upon the table below:

Impact of a 10% Discrepancy on Bits of Guessability

To see how a 10% divergence affects the number of bits of guessability, consider a cryptographic key or password generated with a non-uniform distribution:

Metric	Uniform Distribution (Baseline)	Non-Uniform Distribution (10% Divergence)
Shannon Entropy (H)	10 bits	10 bits (High average randomness)
Min-Entropy (H_{∞})	10 bits	9 bits (10% lower than Shannon)
Max Probability ($p_{\max}$)	$2^{-10} = \frac{1}{1024} \approx 0.098\%$	$2^{-9} = \frac{1}{512} \approx 0.195\%$
First-Guess Success Rate	1 in 1,024 tries	1 in 512 tries (Twice as easy to guess)

Figure 4 In comparison Grover's algorithm reduces the effective security strength (or work required to brute-force) of a symmetric cryptographic key by the same 50% (halving the bit security level) ...

AES as tested, has a 50% to 60% a deviation in values today well in excess of 10% defined herein which demonstrates the proven weakness or deviation from uniform output of the AES key schedule and algorithm design.

3. Additional considerations.

a) Malleability and active attack vectors

Yet another issue has to be considered, namely the message authentication, which is not usually implemented in Vernam Ciphers.

Malleability is common to all stream ciphers (Hanaoka, 2008). Also, the XOR operator is vulnerable to a known-plaintext attack, since $\text{XOR}(\text{plaintext}, \text{ciphertext}) = \text{key}$. Active attackers can also modify messages in transit; a weakness shared with all stream ciphers.

The Medley solution adds proprietary authentication to each key to addresses these well-known issues.

For clarity, there are no Public Key functions involved within any InfoSec or One Cipher use cases.

b) Confused Deputy

Ambient authority attacks, where permissions are attached to the execution context overall identity rather than bound explicitly to each specific request or operation/encryption context. The One Cipher algorithm is an Authenticated Encryption with Associated Data (AEAD) algorithm and hence ties the AAD to the ciphertext. If an attacker intercepts a valid encrypted blob and tries to trick a service (the deputy) into decrypting it using a different user's context or destination, the decryption operation fails before any decryption operation is processed.

c) Malleable ciphers

When a cryptographic system is **malleable** (like the One Cipher or vernan class of ciphers or stream ciphers without a MAC), an attacker can alter the encrypted data. If an attacker knows the structure of the underlying plaintext, they can flip bits to change critical values—such as a user ID from 1002 to 0001 (admin). When the application decrypts this tampered ciphertext, it trusts the result completely. The application (the **deputy**) becomes **confused**. **Standard authenticated encryption (like Encrypt-then-TAG) protects the ciphertext itself from malleability. However, it doesn't protect the context in which that ciphertext is used.** This is where Additional Authenticated Data (AAD) is used in the One Cipher.

d) Side channel/power analysis

The One Cipher or bitwise XOR operation is mathematically perfectly balanced (0 and 1 outputs are equally likely given a random key) and executes in **constant time** on standard processors or physical logic gate implementations. But implementation Hamming Weight/Distance Leakage, logic gate transition period glitches, CPU pipeline and cache-timing side channels and other specific implementation details still require careful consideration.

Factor	Mathematical XOR Cipher	Physical XOR Implementation
Execution Time	Constant (1 cycle)	Constant, but execution pipelines can leak.
Power Consumption	Theoretical zero-variance	High variance due to CMOS switching and glitches.
Data Transmission	Isolated bitwise logic	Leaks via EM radiation and power line fluctuations.

Figure 5 Summary of Vulnerability vs. Mitigation guide

To safely implement the One Cipher in **high-security hardware**, engineers must use physical countermeasures such as **glitch-free dual-rail logic**, **cryptographic masking/blinding** (splitting the secret key into random shares), or **dummy power clamps** to hide the real signal.

Software implementations of the One Cipher make use of specific compiler directives and execute a **masked XOR operation** to prevent power profiles from correlating to the secret key.

e) Lastly Keys

Kerckhoffs's principle, holds that a cryptosystem should be secure, even if everything about the system, except the key, is public knowledge.

Thus, cryptographic keys should generally be generated secretly and uniformly at random in the cryptosystem's key domain (universals); that is in the set of valid keys for the cryptosystem. What makes a key valid depends on the cryptosystem and often parameters (typically including key size).

Since the adversary may know how our system works (Kerckhoffs's principle), we need to find a way to generate keys that don't only depend on a deterministic computation (which the adversary could reproduce). Therefore, all keys must depend on something that the adversary does not know. Such a thing is called a “true” random value.

The proprietary KMS-Medley Key generator (KG) takes a QRNG (TRNG) seed and produces a Cryptographically Secure (non-deterministic) Random Bit Generators (CSNBRNG) in accordance with Shannon min-entropy. Meaning each bit in the key sequence is equally likely to occur.

Medley is based upon a proprietary KG, which generates a CSNBRNG key stream upon demand, the KG TRNG seed is never stored and has a secure ephemeral (one time) existence.

Medley supports user defined KG-universals to enable the creation of arbitrary key domains to suit a wide range of operational deployments of an ITS-KMS.

4.Security Principles

SCI/Codeword or Commercial-Need to Know

The solution supports Top Secret/Sensitive Compartmented Information or commercial need-to-know, cloud-based solutions (hardware or software backed).

The ITS-KMS supports the implementation of Medley 'Universals' which guarantee separation between cryptographic Key Generators, hence ensuring ITS based separation of entangled Key Generators.

All SCI compartments are cryptographically (ITS One Cipher) isolated from all other compartments.

To ensure backward compatibility all existing human based codewords can be utilised to enforce "need to know" requirements for both military and commercial use cases.

5. Cryptographically Secure Keys

Riddle me this?

Can any deterministic function create entropy?

The answer is always no, it cannot...

It is a simple reality, that no computational function can create entropy as it is deterministic or a just a variant of a PRNG source of fixed entropy. A Key Generator is nondeterministic; hence it can create entropy via external real-world entropy. Thus, a Key Generator creates CSNBRNG keys. But the entropy is still limited to the entropy of the real-world source.

Information Theoretic security requires and is underpinned by the existence of a Cryptographic Secure Non-Deterministic Random Bit Stream generator anchored in a real-world entropy.

A Cryptographic Secure Non-Deterministic Random Bit Stream (CSNBRNG) variable length key is not a key or even a QRND number in the typical sense, it must be a cryptographically secure key stream QRNG, which is not derived, via any computational

algorithm. The key stream has to be derived (extracted) from a physical process (in the case for medley from a QRND source).

One cannot compute an algorithm to generate a 'key stream' as the result would be a deterministic and hence represent a predictable PRNG, not a cryptography secure key stream and be a non-deterministic key stream not computed directly from a seed/key.

6. Last Mile Mandatory Security Policy

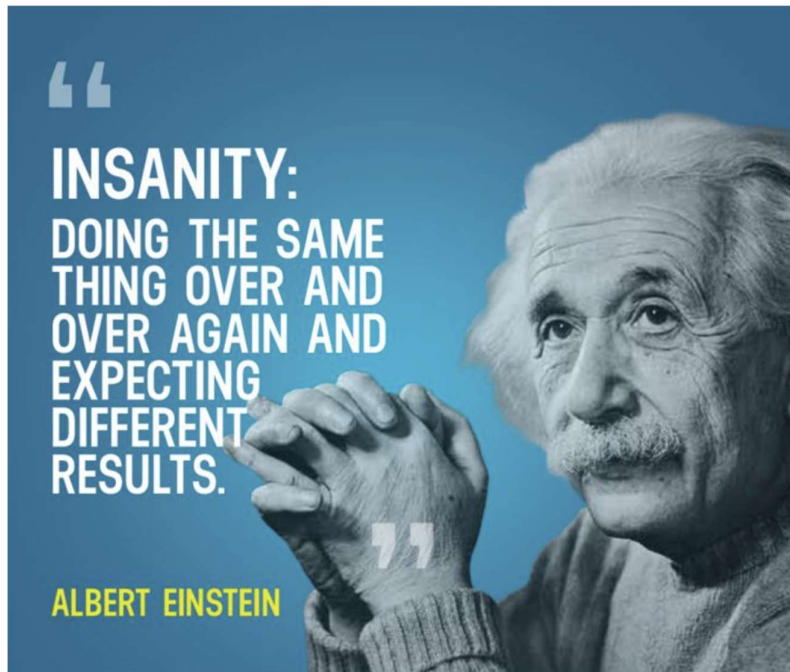
1. All communications Shall be One Cipher (ITS) secured independent of communication media or sChannels security.
2. All InfoSec devices require mandatory biometric authentication, such as Windows Hello fingerprints or Android device fingerprints for InfoSec use cases.
3. All Vaulted Objects shall be end-to-end Once Cipher (device to device) secured before the application of device to host and host One Cipher wrapped.

Conclusion

The Medley (ITS-KMS) creates a pairwise, potentially infinite length, identical secure (verified) bit stream (all bits of the key have equal probability) between two secure Identities (end point devices) according to this proof.



Insanity of Security without a 'Proof of Security'



The quote "Insanity is doing the same thing over and over again and expecting different results" is often attributed to Albert Einstein.

The essence of the quote is to encourage people to be open to change, to learn from past experiences, and to avoid getting stuck in unproductive patterns...

Within cryptography this same insanity of developing algorithms and cipher which come with zero security i.e. doing the same thing over and over again and expecting different results" has become accepted as the norm.

Always Self Verify all cryptographic algorithms and ciphers **before use**, any other form of due diligence is nothing more than pixy dust or magic...

Self-verifying cryptography is the safest mindset when dealing with digital security, never trust 'so called experts' when they come with "zero proof of security" ...

Secure keys?

Grover's is often described as looking for "a needle in a haystack" using quantum mechanics – it finds the needle in roughly the square root of the haystack size, which is vastly faster than checking each piece of hay one by one.

Complexity based algorithms which have no proof-of-security rely on problems that are intractable to brute-force search. Weak or poor-quality keys which do not have Shannon entropy equal to min entropy will typically reduce the key search space by a similar amount as Grover's algorithm and does not require the existence of quantum computer.

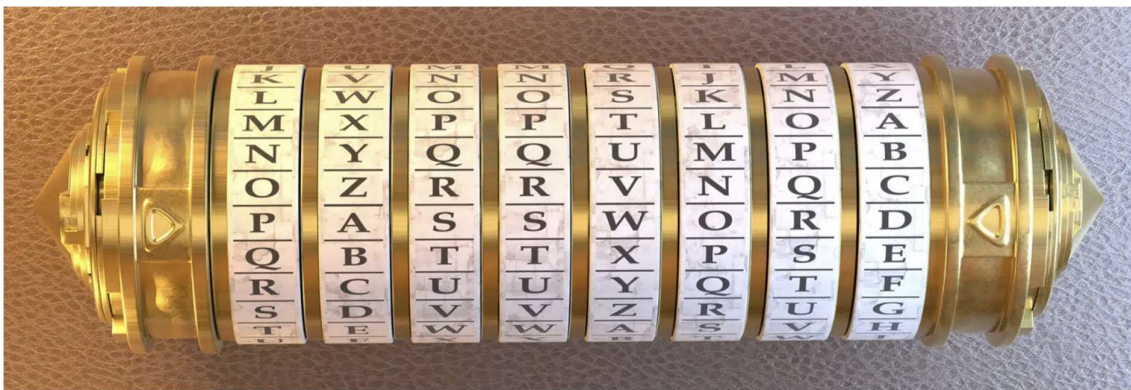


Figure 6 Security is always about entropy, not randomness or key length.

Measured vs Actual Shannon min-entropy

According to the **Laws of Information Theory**, you cannot create *true* entropy out of nothing. Hence no deterministic functions $f(x)$ cannot create or increase entropy, at best that can maintain the input value or decrease it. This proof contains code and AI prompts to measure both Shannon entropy and min-entropy, these need to be used with the following understanding.

Measured entropy can make poorly distributed data *look* perfectly random to statistical tests, the actual cryptographic security (min-entropy) is strictly limited by the uncertainty of the original input. To understand this phenomenon, we must separate how statistical tools measure data from how information theory defines its actual security.

The SOneCipher.GenerateKeyStream output bits are cryptographically scrambled, they possess a near-perfect 50/50 balance of 1s and 0s with no detectable patterns. If you run a Shannon entropy test or a statistical min-entropy estimator on this processed output, the test will be fooled into reporting **maximal entropy** (trending to 8 bits of entropy per output byte).

But actual min-entropy depends entirely on the size of the state space an attacker has to search. Because SOneCipher.GenerateKeyStream function is entirely **deterministic**, they do not introduce new randomness. Only the Medley ->Key Generator, add external randomness to increase Shannon min-entropy.

SOneCipher.GenerateKeyStream function is an excellent **entropy conditioner** (or extractor). It takes poorly distributed, biased real-world entropy and concentrates it into a close to uniform, compact key stream. However, it cannot act as an **entropy generator**. Once you stretch the processed output beyond the bit-strength of the unique input entropy, you are simply stretching a mathematical illusion of entropy or randomness.

The SOneCipher.GenerateKeyStream function hasn't violated information theory by creating new randomness; it has simply taken the original bits of uncertainty and **projected it across a massive computational space**, making it impossible for an observer to predict the next bit without knowing the internal state. It is computationally impossible to figure out what the next bit will be—unless they possess the exact internal state (the seed and current variables) of the generator, successfully "fooling" **Shannon entropy tests** and other statistical suites.

Example

You generate a 128-bit key, but hardware source only chooses from **16 possible variations**. The actual min-entropy of this input is only **4 bits** ($2^4 = 16$).

- You feed this input into the SOneCipher.GenerateKeyStream function and "squeeze" out a 256-bit output.
- The 256-bit output looks completely random. A statistical tool will measure its Shannon entropy and declare it has 256 bits of entropy.
- However, because the function is deterministic, those 16 unique inputs can only ever map to **16 unique outputs**.
- An attacker who knows your system architecture does not have to guess a random 256-bit number. They only have to run the 16 possible inputs through the function to see which output matches yours. Your system still only possesses **4 bits of actual min-entropy**.

Medley Test Results (256-bit keys)

The Medley Key Generator conducts real-time QA on 100% of the Secure Keys used within the One Cipher solution.

```
Debug Trace:
Medley SecureRandom->NextBitTest Pass: Yes
Medley SecureRandom->Has Collisions: No
Medley SecureRandom->Shannon-entropy 5.0000 min-entropy 5.0000
Medley SecureRandom->Miller Madow Entropy 5.4844
```

Acknowledgment

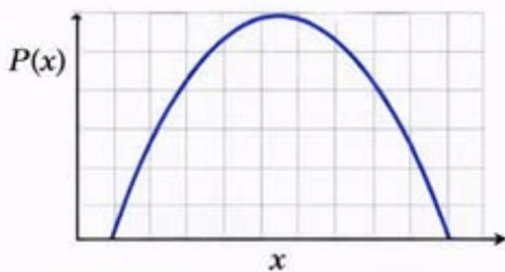
Shannon entropy

$$H(X) = - \sum P(x_i) \log P(x_i)$$

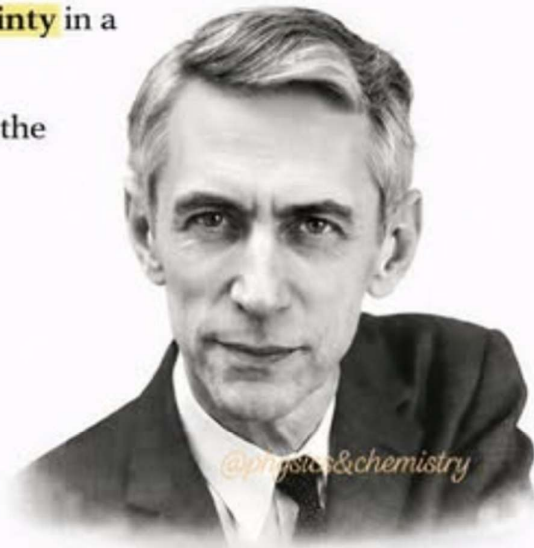
$H(x)$ = entropy $\sum$ = summation $\log$ = logarithm

Entropy measures the **uncertainty** in a random variable X .

- In the formula, $P(x_i)$ represents the **probability** of outcome x_i .



Shannon entropy quantifies the **average amount of information** needed to describe the outcome of a random variable.



Claude Elwood Shannon (1916-2001)

Figure 7 The real hero behind this proof, without who, **verifiable entropic security** would be impossible today.

References

E. Shannon, "Communication theory of secrecy systems," in The Bell System Technical Journal, vol. 28, no. 4, pp. 656-715, Oct. 1949.

E. Shannon, "A mathematical theory of communication," in The Bell System Technical Journal, vol. 27, no. 3, pp. 379-423, July 1948.

Notes

1. Medley is a pairwise, cooperative key generation scheme, the Key Generation Cipher is provably (above) immune from all forms of attack vectors, as the CSNRBG keys are verifiable cryptographically uniformly distributed and hence secure.
2. Medley One cipher, at an abstract level, is essential the XOR (vernan cipher) of two corporately generated Shannon min-entropy bit streams.
3. The Medley protocol produces a set of variable length Medley Vectors; these entangle the Key Generators to produce an identical CSNRBG bit stream (of potentially infinite length).
3. Quantum Key Distribution (QKD) self-deprecates as QKD, requires both participating parties need a) the physical devices, b) a connection between those that allows to run QKD, and c) still some authentication method which either requires a shared secret or some complexity-theoretically secure signature scheme. In short in QKD the two parties have to know each other before communicating and pre-stage shared data first. This defeats the purpose of QKD itself.
4. All modern block ciphers produce deterministic or PRNG key schedules. Such deterministic key schedule, also called a key-schedule algorithm (at best exist as a PRNG computed output), takes the master seed/key and generates a unique set of subkeys or internal state based on a fixed, mathematical process or computable algorithm. This ensures that the same seed/key will always produce the same output, which is a core principle of how most modern and insecure ciphers operate.
5. No $f(x)$ or any computer is capable of producing entropy, as such any seed entropy cannot be increased due to any key schedule expansion.
6. The Medley KG and One Cipher calculates both Shannon entropy and min-entropy in order to determine secure keys.
7. For small sample sizes, Shannon min-entropy calculations should be considered alongside, the corresponding Miller Madow Entropy correction. A **higher Miller-Madow entropy value** means that your dataset contains a higher degree of randomness, uncertainty, or diversity.